ACS101 – PROGRAMMING LANGUAGES

UNIT 4 – PYTHON PROGRAMMING

COMPLETE AKTU EXAM NOTES

Beginner-friendly • Theory • Syntax • Flowcharts • 26 Programs • Question Bank

Prepared as a structured study resource for first-year B.Tech students

Simple English | Exam-oriented answers | Readable code and diagrams

How to use these notes

This document follows the Unit 4 topics and program list supplied in the study request. It explains concepts from the beginning, then gives complete programs and an exam practice bank.

PYQ accuracy note: No authenticated AKTU question-paper PDFs were supplied or verified while preparing this document. Therefore, the question bank is labelled EXPECTED / MODEL QUESTIONS, not actual previous-year questions. No year or session is invented.

- For a 2-mark answer, learn the definition and one key point.
- For a 5-mark answer, write the definition, explanation, syntax and a small example.
- For a 7-mark answer, add an algorithm or flowchart, complete code and output.
- Run each program yourself and change the input values to understand it.

Table of contents

- 1. Introduction to Python
- 2. Syntax, Variables and Data Types
- 3. Input, Output and Operators
- 4. Conditional Statements
- 5. Loops and Control Statements
- 6. Functions in Python
- 7. Complete Python Programs (1–26)
- 8. Flowchart Practice
- 9. Important Difference Tables
- 10. Expected AKTU Question Bank with Answers
- 11. Last-minute Revision and Priority List

1. Introduction to Python

1.1 Definition and introduction

Definition: Python is a high-level, general-purpose, interpreted programming language known for clear and readable syntax.

Explanation: A programmer writes instructions in Python. The Python implementation runs those instructions using an interpreter and supporting runtime components. Python is used for small scripts as well as large applications.

1.2 History

- Python was created by Guido van Rossum.
- Its development began in the late 1980s; it was first released publicly in 1991.
- The name was inspired by the comedy group Monty Python, not by the snake.
- Python 3 is the modern major version used for current learning and development.

1.3 Main features

- Easy-to-read syntax: code is close to plain English.
- High-level: many low-level details are handled by the language/runtime.
- Interpreted execution model: Python programs are commonly run through an interpreter.
- Dynamically typed: a name can refer to values of different types at different times.
- Object-oriented and multi-paradigm: supports procedural, object-oriented and functional styles.
- Portable: the same source code often runs on different operating systems with suitable Python installed.
- Large standard library and third-party ecosystem.
- Free and open source.

1.4 Applications of Python

Area	Typical use
Artificial Intelligence	Building intelligent applications, language and image tools.
Machine Learning	Training and using models with libraries such as scikit-learn and PyTorch.
Data Science	Cleaning, analysing and visualising data with pandas and NumPy.
Web Development	Building server-side websites and APIs with Django, Flask or FastAPI.
Automation	Renaming files, processing spreadsheets, repetitive office tasks and testing.
Scientific computing	Numerical work, simulations and research.

1.5 Advantages and disadvantages

Advantages	Disadvantages / limitations
Readable and beginner-friendly.	Often slower than compiled low-level languages such as C for CPU-heavy tasks.
Large libraries and community.	Can use more memory for some workloads.
Works across operating systems.	Dynamic typing can allow type-related errors to appear at runtime.
Fast development and prototyping.	Not usually the first choice for low-level device drivers or hard real-time systems.
Integrates with other languages.	The Global Interpreter Lock in common CPython builds can limit parallel execution of Python bytecode in threads.

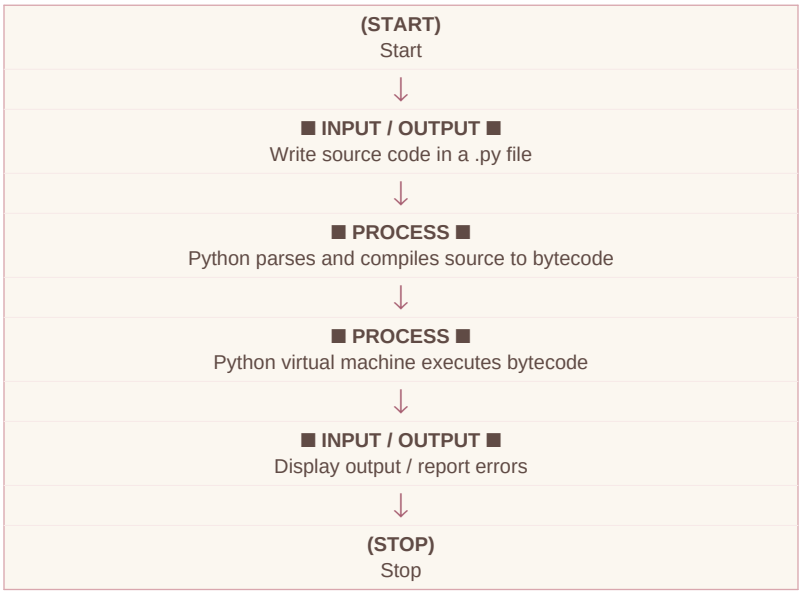
1.6 Python and C

Basis	Python	C
Level	High-level language.	Procedural, general-purpose language with lower-level control.
Typing	Dynamic typing.	Static typing.
Execution	Commonly compiled to bytecode and executed by a runtime/virtual machine.	Usually compiled to native machine code.
Syntax	Short and readable; indentation defines blocks.	Uses braces { } and semicolons.
Memory	Automatic memory management in typical implementations.	Manual allocation/deallocation is available and often programmer-managed.
Speed	Often slower for CPU-bound pure-language code.	Usually faster for comparable low-level tasks.
Use cases	Automation, web, data, AI/ML.	Operating systems, embedded systems, performance-critical software.

1.7 Interpreter and execution process

An interpreter is a program that runs another program. In CPython, source code is parsed and compiled to bytecode; the Python virtual machine executes that bytecode. The simple classroom phrase “Python is interpreted” describes the usual user experience, but the implementation has a compilation-to-bytecode stage.

Basic Python program execution



Working: Save the source file, run it with the Python interpreter, and read the output. Syntax or runtime errors are reported during the process.

2. Python Syntax, Variables and Data Types

2.1 Basic syntax

A Python statement is usually written on its own line. Python uses indentation to show which statements belong to a block.

```
print("Hello, Python!")
a = 10
b = 20
total = a + b
print(total)
```

2.2 Indentation

Indentation means spaces at the beginning of a line. Four spaces are the common convention. Statements in the same block should use the same indentation.

```
age = 18
if age >= 18:
    print("Adult")
    print("Eligible by age")
print("Program finished")
```

Incorrect indentation can cause an IndentationError or change which statements belong to a block. Do not mix tabs and spaces.

2.3 Comments

- A comment explains code and is ignored by Python when executing it.
- A single-line comment begins with #.
- Triple-quoted strings are commonly used for documentation strings (docstrings); they are not technically comment syntax.

```
# This is a single-line comment
name = "Asha"
print(name)

def greet():
    """Return a short greeting (docstring)."""
    return "Hello"
```

2.4 Variables and naming rules

Definition: A variable name is an identifier that refers to an object/value in a program.

- A name can contain letters, digits and underscores, but cannot start with a digit.
- Names are case-sensitive: marks and Marks are different.
- A Python keyword such as if, for or class cannot be used as a variable name.
- Use meaningful names such as total_marks rather than unclear names.

```
student_name = "Riya"
age = 18
marks_1 = 82
print(student_name, age, marks_1)
```

2.5 Dynamic typing and name binding

Python is dynamically typed: the type belongs to the object, and a name can be rebound to an object of another type. Name binding means associating a name with an object.

```
value = 25       # value refers to an integer object
value = "twenty-five"  # now it refers to a string object
print(value)
print(type(value))
```

2.6 Basic data types

Type	Meaning	Example
int	Whole numbers	count = 12
float	Numbers with a decimal part	price = 19.5
str	Text / sequence of characters	city = 'Delhi'
bool	Logical value: True or False	is_valid = True

2.7 type() function

The type() function tells the type/class of an object.

```
a = 10
b = 3.5
c = "Python"
d = True
print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

2.8 Type conversion / type casting

Type conversion changes a value from one data type to another.

- **Implicit conversion:** Python automatically converts compatible values, for example int to float in mixed arithmetic.
- **Explicit conversion:** the programmer requests conversion using functions such as int(), float(), str() and bool().

```
x = 5
y = 2.5
z = x + y
print(z, type(z))       # 7.5, float

text_number = "42"
number = int(text_number)
print(number + 8)       # 50

decimal_text = "3.14"
print(float(decimal_text))
print(str(100))
```

int('3.5') raises ValueError. Convert it with float('3.5') first if a decimal value is intended.

2.9 Static typing and dynamic typing

Static typing	Dynamic typing
A variable's declared type is checked/known through the language's static type system.	A name can refer to objects of different types during execution.
Example: C requires declarations such as int n;	Example: Python allows n = 4 followed by n = 'four'.
Many type errors can be caught before running, depending on compiler/checker.	Many type errors are found while the program runs.

3. Input, Output and Operators

3.1 print() function

print() displays values on the screen. It can print multiple values, and sep and end control separators and line endings.

```
print("Hello")
print("A", "B", "C", sep="-")
print("Same line", end=" ")
print("continues")
```

3.2 input() function

input() displays an optional prompt and reads a line typed by the user. It always returns the entered data as a string.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name)
print("Next year you will be", age + 1)
```

3.3 Taking different types of input

```
whole = int(input("Enter an integer: "))
decimal = float(input("Enter a decimal: "))
word = input("Enter text: ")
print(whole, decimal, word)
```

3.4 Basic input validation

Validate input before using it. A beginner-friendly method is try/except, which handles invalid numeric input without crashing.

```
try:
    marks = float(input("Enter marks (0 to 100): "))
    if 0 <= marks <= 100:
        print("Valid marks:", marks)
    else:
        print("Marks must be from 0 to 100.")
except ValueError:
    print("Please enter a number.")
```

3.5 Operators

Operator group	Operators	Purpose
Arithmetic	+ - * / // % **	Addition, subtraction, multiplication, division, floor division, remainder, power.
Comparison	== != > < >= <=	Compare values; result is True or False.
Logical	and or not	Combine or reverse conditions.
Assignment	= += -= *= /= //= %= **=	Assign or update a variable.

3.6 Operator examples

```
a = 17
b = 5
print(a + b)   # 22
print(a / b)   # 3.4
print(a // b)  # 3
print(a % b)   # 2
print(a ** 2)  # 289
print(a > b)   # True
print(a == b)  # False
print(a > 10 and b < 10)  # True
```

3.7 Precedence and associativity

Precedence decides which operator is evaluated first. Parentheses can make the intended order clear. Operators of equal precedence generally follow their associativity rules.

High to lower (common operators)	Example
Parentheses	(2 + 3) * 4
Exponentiation **	2 ** 3
Unary +, -, not	-x, not ready
*, /, //, %	a * b / c
+ and -	a + b - c
Comparisons	a >= b
not, and, or	a > 0 and b > 0
Assignment	x += 1

Solved expression: 5 + 2 * 3 ** 2 = 5 + 2 * 9 = 5 + 18 = 23. Parentheses override normal precedence.

Associativity example: 20 / 5 * 2 is evaluated left to right: (20 / 5) * 2 = 8. Exponentiation groups right to left: 2 ** 3 ** 2 = 2 ** (3 ** 2) = 512.

4. Conditional Statements

Decision-making allows a program to choose which block to execute based on whether a condition is True or False. A condition is usually a comparison or logical expression.

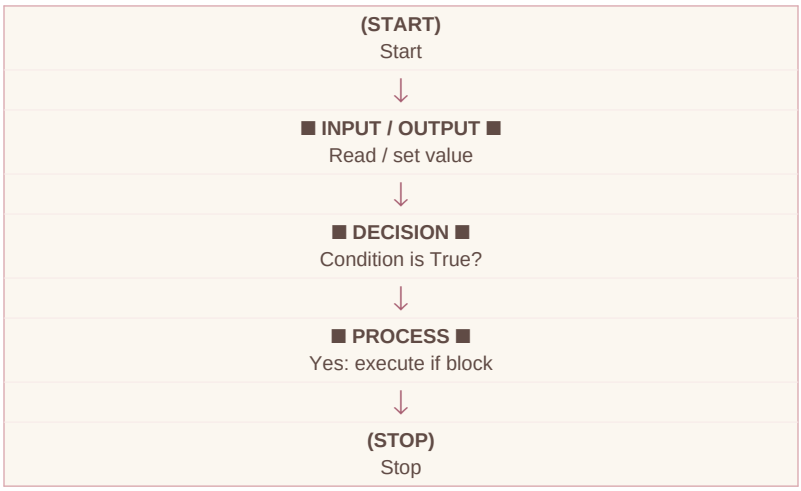
4.1 if statement

The if block runs only when its condition is True.

```
if condition:
    statement(s)
```

```
temperature = 35
if temperature > 30:
    print("It is a hot day")
```

if statement



Working: If the condition is False, control skips the indented block and moves to the next statement.

4.2 if-else statement

```
if condition:
    statement_if_true
else:
    statement_if_false
```

```
number = 8
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

if-else statement



Working: Exactly one of the two branches is selected.

4.3 if-elif-else ladder

```
if condition_1:
    block_1
elif condition_2:
    block_2
else:
    default_block
```

```
marks = 76
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
```

```
    grade = "D"
else:
    grade = "F"
print(grade)
```

if-elif-else ladder



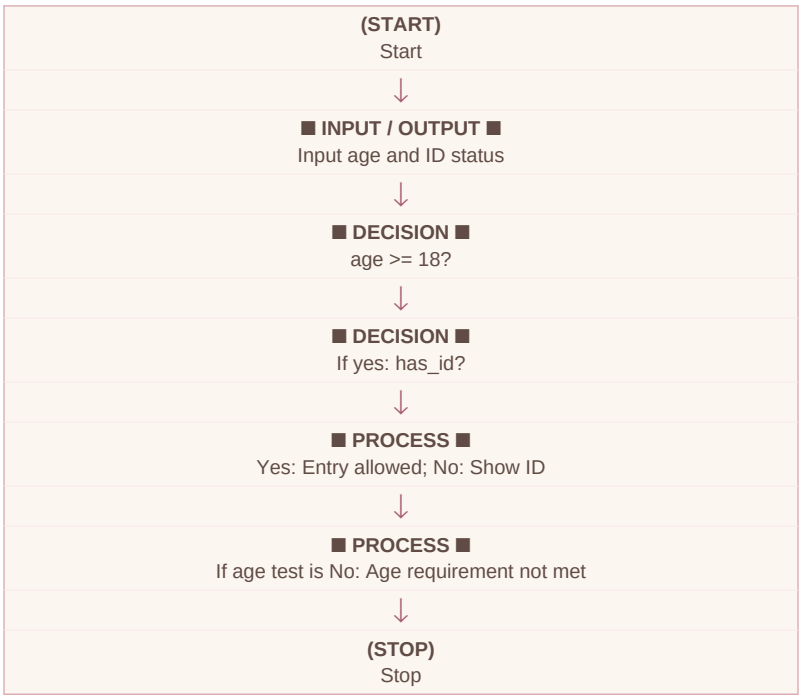
Working: Conditions are checked from top to bottom. The first True branch runs; later branches are skipped.

4.4 Nested if

A nested if is an if statement placed inside another if or else block.

```
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Show your ID")
else:
    print("Age requirement not met")
```

Nested if



Working: The inner decision is reached only when the outer age condition is True.

5. Loops and Control Statements

A loop repeats a block of code. It reduces repeated statements and is useful when processing a sequence or repeating work until a condition changes.

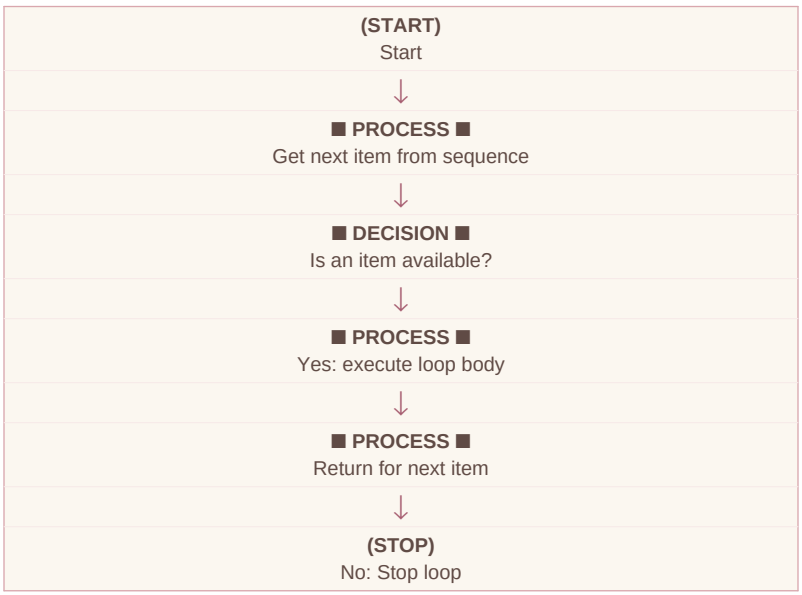
5.1 for loop

A for loop visits each item in an iterable, such as a string, list or range.

```
for item in sequence:
    statements
```

```
for letter in "CAT":
    print(letter)
```

for loop



Working: The loop body runs once for each available item. It ends when no item remains.

5.2 range() function

range() produces a sequence of integers, commonly used with for loops.

Form	Values produced for example
range(5)	0, 1, 2, 3, 4
range(2, 6)	2, 3, 4, 5
range(1, 10, 2)	1, 3, 5, 7, 9

The stop value is not included. The step can be positive or negative, but it must not be zero.

```
for i in range(1, 6):
    print(i)
```

5.3 while loop

A while loop repeats as long as its condition remains True. Update the control variable so that the loop can eventually stop.

```
while condition:
    statements
```

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

while loop



Working: The condition is checked before each repetition. If it is False at the beginning, the body runs zero times.

5.4 Iterating through strings

```
word = "Python"
for character in word:
    print(character)
```

Each repetition assigns the next character to character. Strings are sequences, so they can be traversed with for loops.

5.5 break statement

break immediately ends the nearest enclosing loop.

```
for n in range(1, 6):
    if n == 4:
        break
    print(n)
# Output: 1 2 3
```

5.6 continue statement

continue skips the remaining statements in the current iteration and moves to the next iteration.

```
for n in range(1, 6):
    if n == 3:
        continue
    print(n)
# Output: 1 2 4 5
```

break and continue



Working: break exits the loop; continue skips only the current iteration. Both affect the nearest enclosing loop.

5.7 Nested loops

A nested loop is a loop inside another loop. For each outer-loop iteration, the inner loop completes its repetitions.

```
for row in range(1, 4):
    for column in range(1, 4):
        print("*", end=" ")
    print()
```

5.8 for versus while

for loop	while loop
Best when iterating through a sequence or known range.	Best when repetition depends on a condition.
Gets items from an iterable.	Checks a Boolean condition before each iteration.
Often needs no manual counter update.	Programmer usually updates the control variable.
Example: for i in range(5):	Example: while count < 5:

6. Functions in Python

6.1 Definition and importance

Definition: A function is a named, reusable block of code that performs a particular task. It can accept inputs (arguments) and can return a result.

Functions divide a large program into smaller parts. They reduce repeated code and make testing, reading and maintenance easier.

6.2 Built-in and user-defined functions

Built-in functions	User-defined functions
Already provided by Python.	Created by the programmer using def.
Examples: print(), input(), len(), type(), int().	Example: def add(a, b): return a + b

6.3 Defining and calling a function

```
def greet():
    print("Welcome to Python")

greet()  # function call
```

def begins a function definition. The indented body is stored as the function's work. Calling greet() runs that body.

6.4 Parameters and arguments

A parameter is a name written in the function definition. An argument is the actual value supplied when the function is called.

```
def add(a, b):      # a and b are parameters
    return a + b

answer = add(4, 6)   # 4 and 6 are arguments
print(answer)
```

6.5 Positional arguments

Positional arguments are matched to parameters by their order.

```
def introduce(name, city):
    print(name, "lives in", city)

introduce("Riya", "Delhi")
```

6.6 Default arguments

A default argument has a value in the function definition. The caller may omit that argument.

```
def greet(name, message="Hello"):
    print(message, name)

greet("Asha")
greet("Asha", "Good morning")
```

6.7 return statement

return ends the function call and sends a value back to the caller. A function can return one object; multiple comma-separated values are returned together as a tuple.

```
def square(number):
    return number * number

result = square(5)
print(result)
```

6.8 print() versus return

print()	return
Displays information on the screen.	Sends a result back to the caller.
Does not by itself provide that displayed value to the calling expression.	The returned value can be stored, reused or passed to another function.
A function that only prints and has no return statement returns None.	A return statement can return a number, string, list, etc.

6.9 Understanding None

None is a special Python object that represents the absence of a value. If a function reaches its end without a return statement, Python returns None automatically.

```
def show_message():
    print("Hi")
```

```
result = show_message()
print(result)  # None
```

6.10 Function versus method

Function	Method
A callable block that can be called by its name.	A function associated with an object or class.
Example: len(word).	Example: word.upper().

6.11 Advantages of functions

- Code reuse: write a task once and call it many times.
- Modularity: divide a large program into smaller parts.
- Readability: meaningful function names explain the program structure.
- Testing and debugging: each function can be checked separately.
- Maintenance: a change to one function can update all calls that use it.

7. Complete Python Programs (1–26)

All programs below use Python 3. Sample outputs are illustrative and depend on the sample input shown. For real applications, validate input and handle edge cases.

Program 1 – Hello World

Question: Write a Python program to display Hello World.

Logic / algorithm: Call print() with the required message.

Python code

```
print("Hello World")
```

Sample input: No input required.

Sample output:

```
Hello World
```

Step-by-step explanation: print() displays the text between quotation marks.

Program 2 – Addition of two numbers

Question: Read two numbers and display their sum.

Logic / algorithm: Read two numeric values, add them and print the result.

Python code

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
total = a + b
print("Sum =", total)
```

Sample input: Enter first number: 12\nEnter second number: 8

Sample output:

```
Sum = 20.0
```

Step-by-step explanation: float() accepts whole or decimal input. The + operator adds the values.

Program 3 – Area of a circle

Question: Calculate the area of a circle using its radius.

Logic / algorithm: Use area = pi × radius × radius. Reject a negative radius.

Python code

```
radius = float(input("Enter radius: "))
if radius < 0:
    print("Radius cannot be negative")
else:
    area = 3.14159 * radius ** 2
    print("Area =", area)
```

Sample input: Enter radius: 7

Sample output:

```
Area = 153.93791
```

Step-by-step explanation: ** 2 squares the radius. The constant 3.14159 is an approximation of pi.

Program 4 – Swap two numbers

Question: Swap the values of two variables.

Logic / algorithm: Use Python's multiple assignment to exchange the values.

Python code

```
a = input("Enter a: ")
b = input("Enter b: ")
print("Before swap:", a, b)
a, b = b, a
print("After swap:", a, b)
```

Sample input: Enter a: 10\nEnter b: 20

Sample output:

```
Before swap: 10 20\nAfter swap: 20 10
```

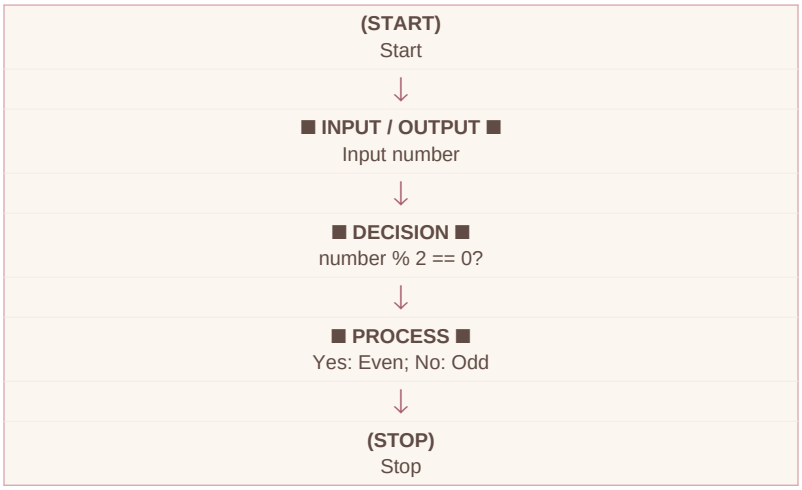
Step-by-step explanation: The right-hand values are evaluated first, then assigned to the names on the left.

Program 5 – Even or odd

Question: Check whether an integer is even or odd.

Logic / algorithm: An integer is even when its remainder after division by 2 is zero.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
number = int(input("Enter an integer: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Sample input: Enter an integer: 14

Sample output:

```
Even
```

Step-by-step explanation: % gives the remainder. The if-else statement selects one result.

Program 6A – Largest of two numbers

Question: Find the largest of two numbers.

Logic / algorithm: Compare the two values and handle equality.

Python code

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
if a > b:
    print("Largest =", a)
elif b > a:
    print("Largest =", b)
else:
    print("Both are equal")
```

Sample input: Enter first number: 9\nEnter second number: 12

Sample output:

```
Largest = 12.0
```

Step-by-step explanation: The first condition checks whether a is greater. The second checks whether b is greater.

Program 6B – Largest of three numbers

Question: Find the largest of three numbers.

Logic / algorithm: Check whether a is at least as large as both others, then b, otherwise c.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print("Largest =", largest)
```

Sample input: Enter first number: 8\nEnter second number: 15\nEnter third number: 11

Sample output:

```
Largest = 15
```

Step-by-step explanation: and requires both comparisons to be True. >= also handles tied maximum values.

Program 7 – Positive, negative or zero

Question: Classify a number as positive, negative or zero.

Logic / algorithm: Compare the number with zero using if-elif-else.

Python code

```
number = float(input("Enter a number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Sample input: Enter a number: -3

Sample output:

```
Negative
```

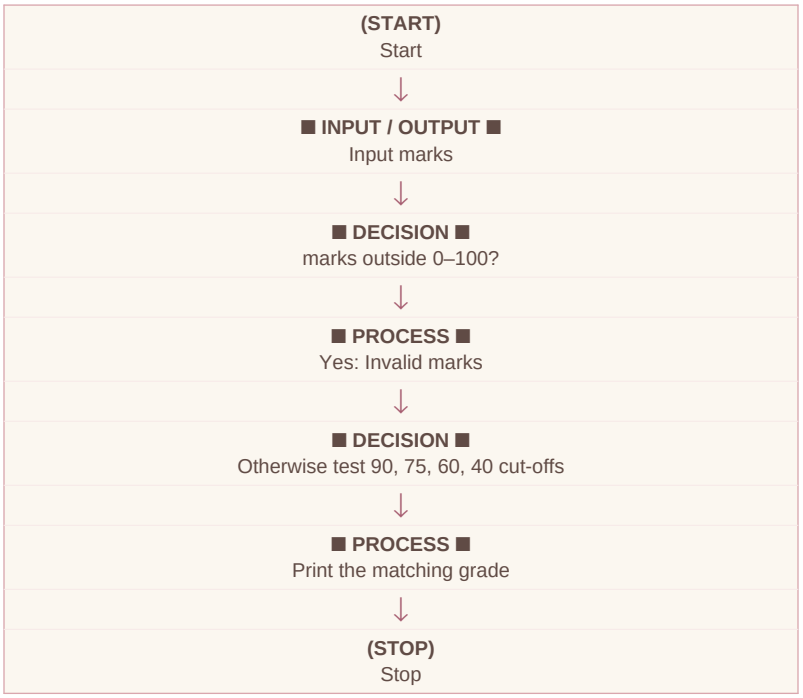
Step-by-step explanation: Only one of the three mutually exclusive branches is executed.

Program 8 – Marks-to-grade calculator

Question: Assign a grade for marks from 0 to 100.

Logic / algorithm: Validate the range first, then test grade boundaries from highest to lowest.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
marks = float(input("Enter marks (0-100): "))
if marks < 0 or marks > 100:
    print("Invalid marks")
elif marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
elif marks >= 40:
    print("Grade D")
else:
    print("Grade F")
```

Sample input: Enter marks (0-100): 82

Sample output:

```
Grade B
```

Step-by-step explanation: The range check prevents invalid marks. Conditions are checked from top to bottom.

Program 9 – Leap year

Question: Check whether a year is a leap year.

Logic / algorithm: A year is leap if divisible by 400, or divisible by 4 but not by 100.

Python code

```
year = int(input("Enter year: "))
```

```
if year % 400 == 0:
    print("Leap year")
elif year % 100 == 0:
    print("Not a leap year")
elif year % 4 == 0:
    print("Leap year")
else:
    print("Not a leap year")
```

Sample input: Enter year: 2024

Sample output:

```
Leap year
```

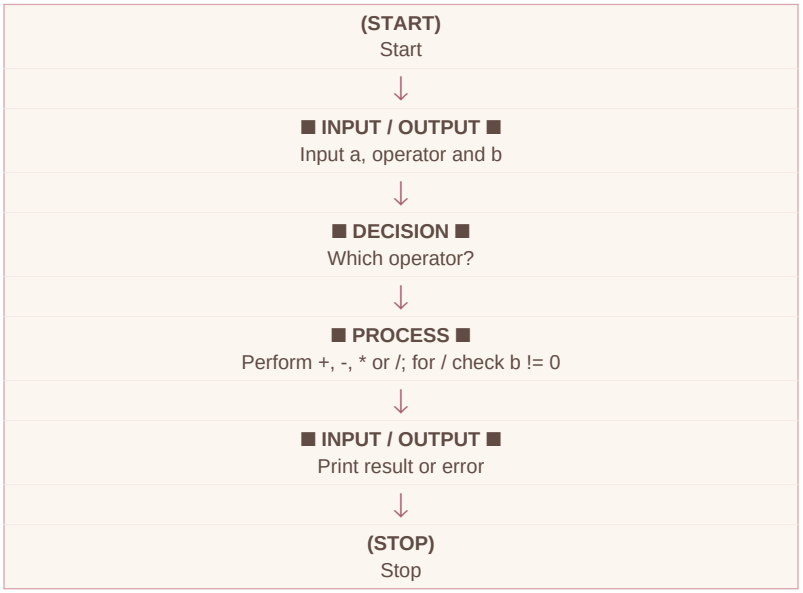
Step-by-step explanation: The order matters: century years are treated separately. For example, 1900 is not leap, but 2000 is leap.

Program 10 – Simple calculator

Question: Perform addition, subtraction, multiplication or division based on the selected operator.

Logic / algorithm: Read two numbers and an operator; choose the operation. Check division by zero.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
a = float(input("Enter first number: "))
op = input("Enter operator (+, -, *, /): ")
b = float(input("Enter second number: "))
if op == "+":
    print("Result =", a + b)
elif op == "-":
    print("Result =", a - b)
elif op == "*":
    print("Result =", a * b)
elif op == "/":
    if b != 0:
        print("Result =", a / b)
    else:
        print("Cannot divide by zero")
else:
    print("Invalid operator")
```

Sample input: Enter first number: 12\nEnter operator (+, -, *, /): *\nEnter second number: 4

Sample output:

```
Result = 48.0
```

Step-by-step explanation: The outer if-elif ladder selects the operator. The nested if prevents division by zero.

Program 11 – Multiplication table

Question: Print the multiplication table of a number from 1 to 10.

Logic / algorithm: Repeat i from 1 through 10 and print number × i.

Python code

```
number = int(input("Enter a number: "))
for i in range(1, 11):
    print(number, "x", i, "=", number * i)
```

Sample input: Enter a number: 5

Sample output:

```
5 x 1 = 5\n5 x 2 = 10\n...\n5 x 10 = 50
```

Step-by-step explanation: range(1, 11) includes 1 through 10 because the stop value 11 is excluded.

Program 12 – Sum of first N natural numbers

Question: Calculate 1 + 2 + ... + N.

Logic / algorithm: Initialize total to zero and add each integer from 1 to N.

Python code

```
n = int(input("Enter N: "))
total = 0
if n < 1:
    print("Enter a positive integer")
else:
    for i in range(1, n + 1):
        total += i
    print("Sum =", total)
```

Sample input: Enter N: 5

Sample output:

```
Sum = 15
```

Step-by-step explanation: total += i means total = total + i. For N=5, the total is 1+2+3+4+5.

Program 13 – Factorial

Question: Find the factorial of a non-negative integer.

Logic / algorithm: Factorial starts at 1 and multiplies all integers from 1 to N. By definition, 0! = 1.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Factorial is not defined for negative integers")
else:
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    print("Factorial =", fact)
```

Sample input: Enter a non-negative integer: 5

Sample output:

```
Factorial = 120
```

Step-by-step explanation: For 5, fact becomes 1×2×3×4×5 = 120. range(1, 1) is empty for n=0, so fact remains 1.

Program 14 – Fibonacci series

Question: Print the first N terms of the Fibonacci series.

Logic / algorithm: Start with 0 and 1. Each next term is the sum of the previous two.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
n = int(input("Enter number of terms: "))
a, b = 0, 1
if n <= 0:
    print("Enter a positive number of terms")
else:
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b
```

Sample input: Enter number of terms: 7

Sample output:

```
0 1 1 2 3 5 8
```

Step-by-step explanation: At each iteration, print a and then update the pair. The assignment uses the old values to compute the new pair.

Program 15 – Prime number check

Question: Check whether a positive integer is prime.

Logic / algorithm: A prime number is greater than 1 and has exactly two positive factors. Test divisors from 2 up to the square root of n.

Flowchart



Working: Follow the arrows from Start to Stop. A decision diamond selects the correct branch.

Python code

```
n = int(input("Enter an integer: "))
if n <= 1:
    print("Not prime")
else:
    is_prime = True
    for divisor in range(2, int(n ** 0.5) + 1):
        if n % divisor == 0:
            is_prime = False
            break
    if is_prime:
        print("Prime")
    else:
        print("Not prime")
```

Sample input: Enter an integer: 29

Sample output:

```
Prime
```

Step-by-step explanation: If any divisor divides n exactly, it is not prime and break ends the search. Checking only up to square root is sufficient because factors occur in pairs.

Program 16 – Reverse a number

Question: Reverse the digits of a non-negative integer.

Logic / algorithm: Take the last digit using % 10, append it to the reversed number, then remove the last digit using // 10.

Python code

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Enter a non-negative integer")
else:
    original = n
    reverse = 0
    while n > 0:
        digit = n % 10
        reverse = reverse * 10 + digit
        n = n // 10
```

```
    print("Reverse of", original, "=", reverse)
```

Sample input: Enter a non-negative integer: 4821

Sample output:

```
Reverse of 4821 = 1284
```

Step-by-step explanation: For 4821, digits are extracted as 1, 2, 8, 4 and appended to reverse. Zero produces reverse 0.

Program 17 – Palindrome number

Question: Check whether a number reads the same forward and backward.

Logic / algorithm: Reverse the number and compare it with the original.

Python code

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Not a palindrome")
else:
    original = n
    reverse = 0
    while n > 0:
        digit = n % 10
        reverse = reverse * 10 + digit
        n //= 10
    if original == reverse:
        print("Palindrome")
    else:
        print("Not a palindrome")
```

Sample input: Enter a non-negative integer: 1221

Sample output:

```
Palindrome
```

Step-by-step explanation: The original value is saved because n is reduced during digit extraction. A number such as 1221 equals its reverse.

Program 18 – Sum of digits

Question: Find the sum of all digits of a non-negative integer.

Logic / algorithm: Repeatedly extract the last digit and add it to total.

Python code

```
n = int(input("Enter a non-negative integer: "))
if n < 0:
    print("Enter a non-negative integer")
else:
    total = 0
    while n > 0:
        digit = n % 10
        total += digit
        n //= 10
    print("Sum of digits =", total)
```

Sample input: Enter a non-negative integer: 583

Sample output:

```
Sum of digits = 16
```

Step-by-step explanation: The digits are 3, 8 and 5; their sum is 16. For input 0, the total remains 0.

Program 19 – Pattern using nested loops

Question: Print a right-angled triangle of stars.

Logic / algorithm: The outer loop controls rows. The inner loop prints the required number of stars in each row.

Python code

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    for j in range(i):
        print("*", end=" ")
```

```
    print()
```

Sample input: Enter number of rows: 4

Sample output:

```
*\n* *\n* * *\n* * * *
```

Step-by-step explanation: For row i, the inner loop runs i times. print() moves to the next line after each row.

Program 20 – Traverse a string

Question: Print each character of a string with its position.

Logic / algorithm: Use enumerate() to get each character and its zero-based index.

Python code

```
text = input("Enter a word: ")
for index, character in enumerate(text):
    print(index, character)
```

Sample input: Enter a word: CODE

Sample output:

```
0 C\n1 O\n2 D\n3 E
```

Step-by-step explanation: enumerate() supplies an index and the corresponding character on each iteration.

Program 21 – BMI calculator

Question: Calculate BMI from weight in kilograms and height in metres.

Logic / algorithm: BMI = weight / height². Check that weight and height are positive.

Python code

```
weight = float(input("Enter weight in kg: "))
height = float(input("Enter height in metres: "))
if weight <= 0 or height <= 0:
    print("Weight and height must be positive")
else:
    bmi = weight / (height ** 2)
    print("BMI =", round(bmi, 2))
```

Sample input: Enter weight in kg: 60\nEnter height in metres: 1.65

Sample output:

```
BMI = 22.04
```

Step-by-step explanation: The program demonstrates a formula and basic input validation. BMI is a screening calculation, not a diagnosis or a complete measure of health.

Program 22 – Temperature converter

Question: Convert Celsius to Fahrenheit or Fahrenheit to Celsius.

Logic / algorithm: Use F = (C × 9/5) + 32 and C = (F – 32) × 5/9.

Python code

```
choice = input("Type C for Celsius to Fahrenheit or F for Fahrenheit to Celsius:
    ").upper()
if choice == "C":
    celsius = float(input("Enter Celsius: "))
    fahrenheit = (celsius * 9 / 5) + 32
    print("Fahrenheit =", fahrenheit)
elif choice == "F":
    fahrenheit = float(input("Enter Fahrenheit: "))
    celsius = (fahrenheit - 32) * 5 / 9
    print("Celsius =", celsius)
else:
    print("Invalid choice")
```

Sample input: Type C for Celsius to Fahrenheit or F for Fahrenheit to Celsius: C\nEnter Celsius: 25

Sample output:

```
Fahrenheit = 77.0
```

Step-by-step explanation: upper() makes the choice case-insensitive. The selected formula converts the entered temperature.

Program 23 – Function-based calculator

Question: Create a calculator using a user-defined function.

Logic / algorithm: Pass two numbers and an operator to a function; return the result or an error message.

Python code

```
def calculate(a, b, operator):
    if operator == "+":
        return a + b
    elif operator == "-":
        return a - b
    elif operator == "*":
        return a * b
    elif operator == "/":
        if b == 0:
            return "Cannot divide by zero"
        return a / b
    else:
        return "Invalid operator"

x = float(input("First number: "))
op = input("Operator (+, -, *, /): ")
y = float(input("Second number: "))
print("Result =", calculate(x, y, op))
```

Sample input: First number: 9\nOperator (+, -, *, /): +\nSecond number: 6

Sample output:

```
Result = 15.0
```

Step-by-step explanation: calculate() receives three arguments. Each branch returns a result; the final print displays the returned value.

Program 24 – Function to find factorial

Question: Write a function that returns the factorial of a number.

Logic / algorithm: Define factorial(n), validate n, multiply values in a loop and return the result.

Python code

```
def factorial(n):
    if n < 0:
        return None
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

number = int(input("Enter a non-negative integer: "))
answer = factorial(number)
if answer is None:
    print("Invalid input")
else:
    print("Factorial =", answer)
```

Sample input: Enter a non-negative integer: 4

Sample output:

```
Factorial = 24
```

Step-by-step explanation: The function returns None for a negative input. Otherwise it returns the calculated factorial. The caller checks the result.

Program 25 – Function to generate Fibonacci series

Question: Write a function that returns the first N Fibonacci terms.

Logic / algorithm: Build a list, repeatedly append the current term, and update the two values.

Python code

```
def fibonacci(n):
```

```
    terms = []
    a, b = 0, 1
    for i in range(n):
        terms.append(a)
        a, b = b, a + b
    return terms

count = int(input("Number of terms: "))
if count < 0:
    print("Enter zero or a positive number")
else:
    print(fibonacci(count))
```

Sample input: Number of terms: 6

Sample output:

```
[0, 1, 1, 2, 3, 5]
```

Step-by-step explanation: append() adds an item to the list. The function returns the complete list after the loop.

Program 26 – Function with default arguments

Question: Create a greeting function with a default message.

Logic / algorithm: Give message a default value. Call the function once with the default and once with a supplied message.

Python code

```
def greet(name, message="Hello"):
    return message + ", " + name + "!"

print(greet("Bhumika"))
print(greet("Aman", "Good morning"))
```

Sample input: No input required.

Sample output:

```
Hello, Bhumika!\nGood morning, Aman!
```

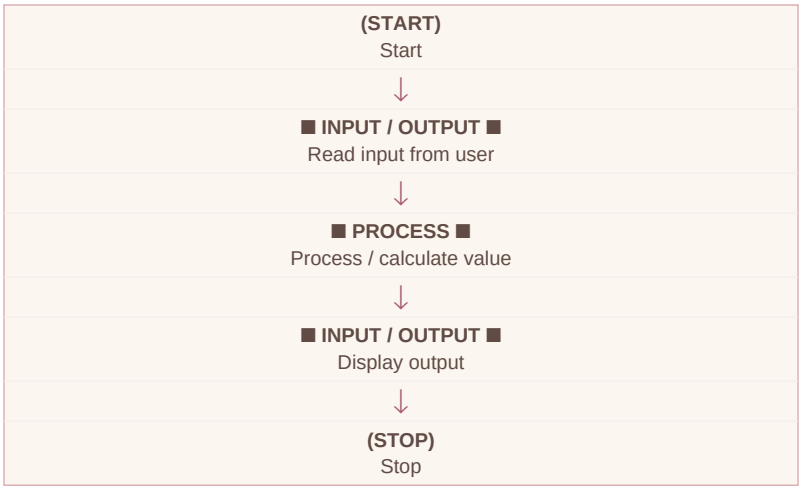
Step-by-step explanation: When the second argument is omitted, message is 'Hello'. Supplying a second argument replaces the default.

8. Flowchart Practice and Algorithm Writing

A flowchart is a diagram that shows the steps of an algorithm using standard symbols. In these notes, the flowcharts are arranged vertically so they remain readable on A4 pages.

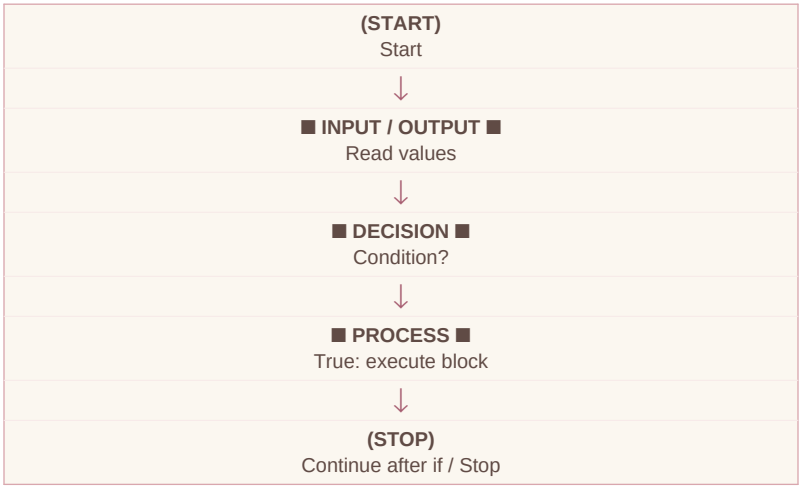
Symbol	Meaning	Use
Oval	Start / Stop	Beginning or end of a process.
Parallelogram	Input / Output	Read input or display output.
Rectangle	Process	Calculation or assignment.
Diamond	Decision	Test a condition; branches are Yes/No.
Arrow	Flow line	Shows direction of control.

Input and output process



Working: Input is read, processed and then displayed.

if statement



Working: False skips the block.

if-else statement



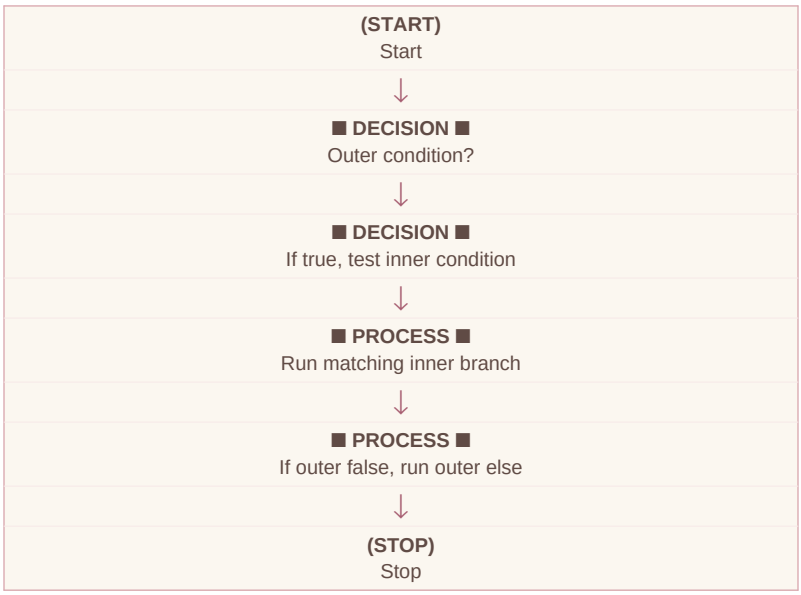
Working: Only one branch is executed.

if-elif-else ladder



Working: Conditions are checked in order; the first True condition wins.

Nested if



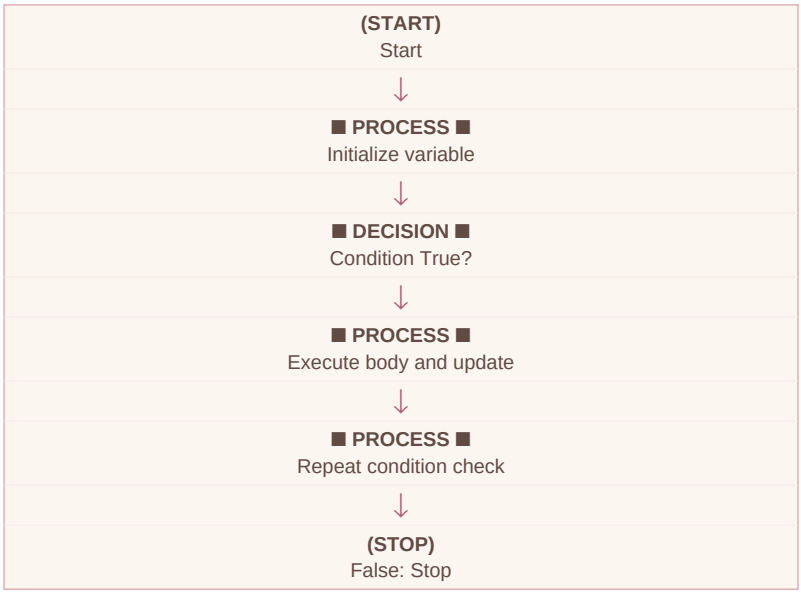
Working: The inner decision is inside the outer branch.

for loop



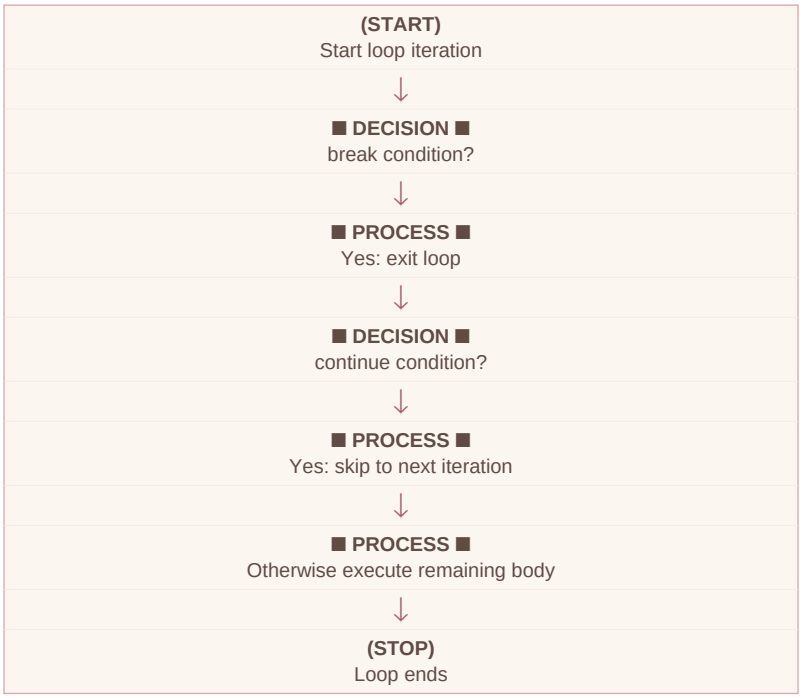
Working: A for loop visits each item in the iterable.

while loop



Working: The condition is checked before each iteration.

break and continue



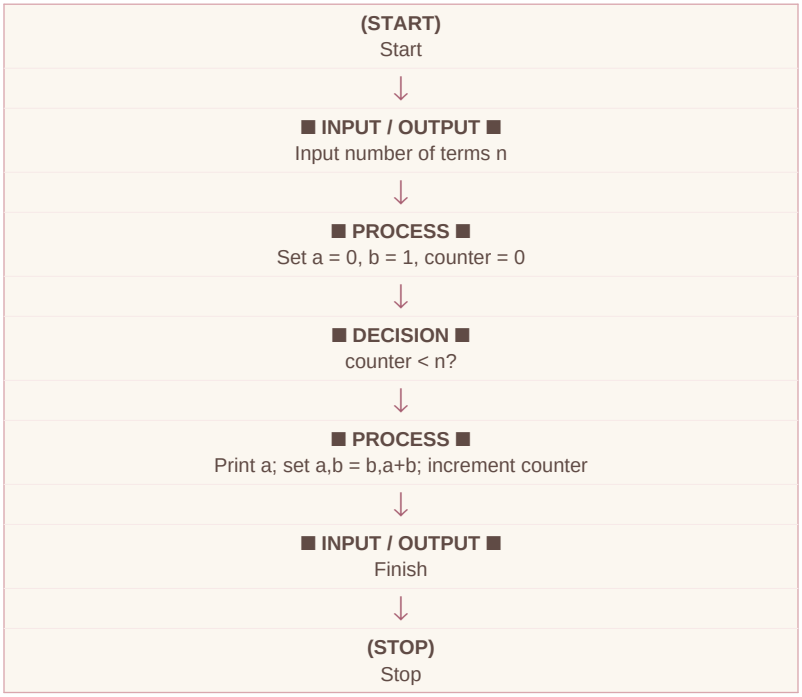
Working: break exits; continue skips the current iteration.

Factorial program



Working: Repeat multiplication until i becomes greater than n. For n=0, the result remains 1.

Fibonacci series



Working: The pair is updated after printing each term.

Prime number checking



Working: A factor greater than the square root would have a paired factor smaller than the square root.

Even / odd checking



Working: Remainder zero means the number is divisible by 2.

Largest of three numbers



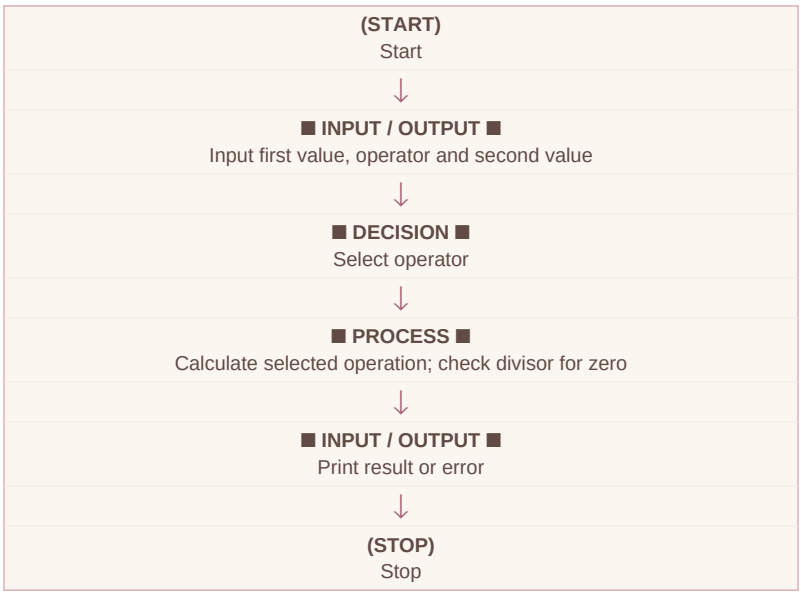
Working: The two comparisons identify which value is at least as large as both others.

Marks-to-grade program



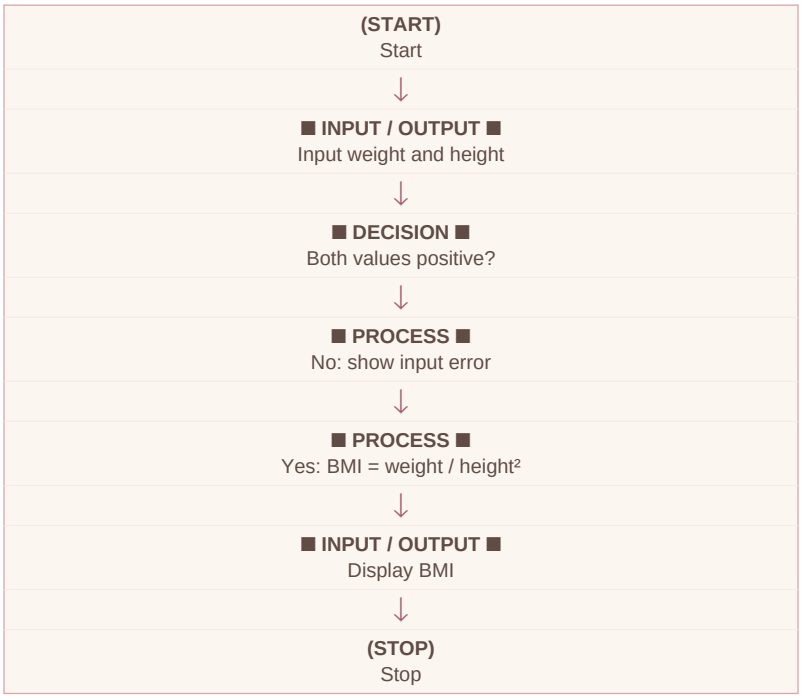
Working: Validation comes first, followed by the ordered grade checks.

Simple calculator



Working: The operator determines which arithmetic expression is evaluated.

BMI calculator



Working: The program validates positive values before calculating the formula.

9. Important Difference Tables

Python vs C	Python	C
Typing	Dynamic.	Static.
Execution	Bytecode plus runtime in CPython.	Typically compiled to native code.
Block structure	Indentation.	Braces.
Memory	Automatic memory management.	Programmer has more direct control.
Typical use	Automation, web, data, AI.	Systems, embedded, performance-sensitive work.

Compiler vs Interpreter	Compiler	Interpreter
Basic role	Translates source code into another form, commonly machine code.	Executes a program through an interpreter/runtime.
Typical workflow	Often produces an executable or object code before running.	Often runs through a runtime when the program is launched.
Error reporting	May report compilation errors before execution.	May report errors as the relevant code is executed.
Note	Modern tools can combine compilation and runtime techniques.	Python implementations may compile source to bytecode before execution.

Static vs Dynamic typing	Static typing	Dynamic typing
Type checking	Type relationships are checked by the language/compiler or static tools before execution.	Types are associated with runtime objects; errors can appear during execution.
Example	C: int age = 18;	Python: age = 18; age = 'eighteen'

int vs float	int	float
Meaning	Whole number.	Floating-point number.
Example	7	7.25
Use	Counting, indexing.	Measurements and decimal calculations.

Implicit vs explicit conversion	Implicit	Explicit
Who performs it	Python automatically converts in a compatible operation.	Programmer calls a conversion function.
Example	5 + 2.5 gives 7.5.	int('12') gives 12.

if-else vs if-elif-else	if-else	if-elif-else
Branches	Two alternatives.	Several alternatives plus optional default.
Use	One condition with True/False result.	Multiple conditions / ranges.
Execution	One of two branches runs.	First True branch runs.

for vs while	for	while
Control	Iterates over an iterable.	Repeats while condition is True.
Best for	Known sequence or range.	Condition-controlled repetition.
Update	Often automatic through next item.	Control variable often updated manually.

break vs continue	break	continue
Effect	Terminates the nearest loop.	Skips the rest of current iteration.
Next action	Control moves after the loop.	Control moves to next iteration check/item.

Function vs method	Function	Method
Meaning	Reusable callable block.	Callable associated with an object or class.
Example	len(name)	name.upper()

Parameter vs argument	Parameter	Argument
Meaning	Name in function definition.	Value supplied in a function call.
Example	x in def square(x):	5 in square(5)

print() vs return	print()	return
Purpose	Displays output.	Sends a value to caller.
Can be reused as result?	Displayed text is not automatically returned.	Returned value can be stored and reused.
Default function result	print() itself returns None.	A function without explicit return also returns None.

Built-in vs user-defined	Built-in	User-defined
Origin	Provided by Python.	Written by programmer.
Example	len(), type(), print().	def area(r): ...

10. Expected AKTU Question Bank with Answers

These are syllabus-based model / expected questions for practice. They are not represented as verified actual AKTU PYQs. Check your official course file and authenticated university papers for session-specific PYQs.

Section A – 2 Marks

1. Define Python.

Answer: Python is a high-level, general-purpose programming language with readable syntax. It is used in areas such as automation, web development and data science.

2. Who developed Python?

Answer: Python was created by Guido van Rossum and first released publicly in 1991.

3. What is an interpreter?

Answer: An interpreter is a program or runtime system that executes instructions of another programming language.

4. Define variable.

Answer: A variable name is an identifier that refers to an object/value in a program.

5. What is dynamic typing?

Answer: Dynamic typing means a name can refer to objects of different data types during program execution.

6. Name four basic Python data types.

Answer: int, float, str and bool.

7. What is indentation?

Answer: Indentation is whitespace at the beginning of a line. Python uses it to define code blocks.

8. What does input() return?

Answer: input() returns the entered line as a string.

9. What is type casting?

Answer: Type casting is converting a value from one data type to another, for example int('5').

10. What is the use of range()?

Answer: range() generates a sequence of integers, commonly used to control for-loop repetitions.

11. Define break.

Answer: break immediately terminates the nearest enclosing loop.

12. Define continue.

Answer: continue skips the remaining statements in the current iteration and starts the next iteration.

13. What is a function?

Answer: A function is a named, reusable block of code designed to perform a task.

14. What is a parameter?

Answer: A parameter is a variable written in a function definition to receive an argument.

15. What is None?

Answer: None is a special Python object representing the absence of a value.

16. What is a default argument?

Answer: A default argument is a parameter that has a predefined value used when the caller omits it.

Section A – Output prediction

17. Find the output.

```
x = 4\nprint(x ** 2 + 1)
```

Answer: 17

18. Find the output.

```
print(10 // 3, 10 % 3)
```

Answer: 3 1

19. Find the output.

```
for i in range(2, 5):\n    print(i, end=' ')
```

Answer: 2 3 4

20. Find the output.

```
x = '5'\nprint(x * 2)
```

Answer: 55

21. Find the output.

```
def add(a, b=2):\n    return a + b\nprint(add(5))
```

Answer: 7

22. Find the output.

```
print(bool(0), bool(1))
```

Answer: False True

Section B – 5 Marks

1. Explain the features and applications of Python.

Answer: Features: readable syntax, high-level, dynamic typing, portability, multi-paradigm support and large library ecosystem. Applications include AI/ML, data science, web development, automation, scientific computing and testing. Give one suitable example for each application.

2. Explain Python data types with examples.

Answer: int stores whole numbers (a=10); float stores decimal values (b=2.5); str stores text (name='Riya'); bool stores True or False (valid=True). Use type(a) to inspect an object's type.

3. Explain implicit and explicit type conversion.

Answer: Implicit conversion is automatic, e.g. 5 + 2.5 produces 7.5. Explicit conversion is requested by the programmer, e.g. int('25'), float('3.2') and str(100). Invalid conversions can raise ValueError.

4. Explain if, if-else and if-elif-else.

Answer: if executes a block when one condition is True. if-else selects one of two branches. if-elif-else tests several conditions in order and executes the first matching branch. Add syntax and a marks-grade example.

5. Differentiate for and while loops.

Answer: for iterates over an iterable and is suitable for sequences/ranges. while repeats as long as a condition is True and is useful for condition-based repetition. A while loop generally needs a variable update.

6. Explain break and continue with examples.

Answer: break exits the nearest loop. continue skips the current iteration and proceeds to the next. Show a for loop with break at 4 and another with continue at 3.

7. Explain functions and their advantages.

Answer: A function is a reusable named code block defined using def. It can take parameters and return a result. Advantages: reusability, modularity, readability, easier testing and maintenance.

8. Explain input() and print().

Answer: input() reads a line and returns a string; convert it with int() or float() when needed. print() displays values and supports sep and end. Give a program that reads two integers and prints their sum.

9. Explain operator precedence with an example.

Answer: Precedence decides the order of operations. Parentheses have priority, followed by exponentiation, unary operators, multiplication/division, addition/subtraction, comparisons and logical operators. Example: 5 + 2 * 3 ** 2 = 23.

10. Explain default and positional arguments.

Answer: Positional arguments match parameters by order. A default argument has a value in the function definition and is used when omitted. Example: def greet(name, message='Hello'): ...

Section C – 7 Marks

1. Explain Python execution and compare Python with C.

Answer outline: Python source is parsed and commonly compiled to bytecode by CPython; the Python virtual machine executes the bytecode. Compare level, typing, syntax, execution model, memory handling, speed and common applications using the table in Section 9.

2. Explain conditional statements with syntax and flowcharts.

Answer outline: Describe if, if-else, if-elif-else and nested if. Explain that conditions produce True/False values and only the selected block executes. Draw the matching flowcharts in Section 8 and include the marks-grade program.

3. Explain loops in Python with examples.

Answer outline: Define iteration. Explain for, while, range(), nested loops, break and continue. Include syntax, flowcharts and a multiplication-table or sum program. State that break exits and continue skips the current iteration.

4. Write a program to find factorial and explain it with an algorithm / flowchart.

Answer outline: Use Program 13. Algorithm: read n; if n is negative report invalid; set fact=1; multiply fact by i for i=1 to n; print fact. For n=0, output is 1.

5. Write a program to generate Fibonacci series.

Answer outline: Use Program 14 or 25. Start a=0, b=1; repeat n times: print a, then update a,b=b,a+b. Explain the first terms 0,1,1,2,3,5.

6. Explain functions, parameters, arguments and return statement.

Answer outline: Define functions with def; explain call, parameters and arguments, positional/default arguments and return. Contrast print and return and explain implicit None. Include a function-based calculator.

7. Write a program to check prime number.

Answer outline: Use Program 15. A prime number is an integer greater than 1 with exactly two positive factors. Test divisors from 2 to floor(sqrt(n)); if any divides evenly, it is not prime.

8. Write a program to find the largest of three numbers using conditional statements.

Answer outline: Use Program 6B. Read a,b,c; test a against b and c; else test b against a and c; otherwise c is largest. Explain use of and and >=.

9. Write a program to calculate BMI and explain input validation.

Answer outline: Use Program 21. Read weight and height; ensure both are positive; calculate weight/(height*height); display rounded result. State that BMI is only a screening calculation.

10. Write a simple calculator using if-elif-else.

Answer outline: Use Program 10. Read two operands and operator; select arithmetic operation; check division by zero; handle invalid operator.

11. Last-minute Revision and Priority List

Very Important – revise first

- Python definition, features, applications and Python vs C.
- Variables, dynamic typing, int/float/str/bool, type() and type conversion.
- input(), print(), arithmetic/comparison/logical operators and precedence.
- if-else, if-elif-else and nested if; marks-to-grade and largest-of-three programs.
- for, while, range(), break and continue; factorial, Fibonacci, prime and multiplication table.
- Function definition, parameters/arguments, positional/default arguments, return and None.
- Flowcharts: if-else, loop, factorial, Fibonacci, prime, largest of three and calculator.

Important – practise next

- Leap year, even/odd, positive/negative/zero and sum of digits.
- Nested-loop star patterns and string traversal.
- Compiler vs interpreter; static vs dynamic typing; function vs method.
- Input validation, temperature converter, BMI formula and function-based calculator.

Additional Practice

- Write a program to count vowels in a string.
- Write a program to find the sum of even numbers from 1 to N.
- Write a function that returns the larger of two values.
- Predict the output of code using nested loops and continue.
- Modify the calculator to support floor division and modulus.

Most important definitions – quick recall

Term	One-line recall
Python	High-level, general-purpose programming language.
Variable	Name referring to an object/value.
Indentation	Leading whitespace used to define Python blocks.
Dynamic typing	A name may refer to objects of different types at runtime.
Type casting	Conversion from one data type to another.
Loop	Control structure that repeats a block.
break	Exits the nearest loop.
continue	Skips the current iteration.
Function	Named reusable block of code.
Parameter	Name in a function definition.
Argument	Value supplied in a function call.
return	Ends a function call and sends a value to its caller.
None	Special object representing no value.

Most important programs – checklist

- Even / odd
- Largest of three

- Marks-to-grade
- Leap year
- Simple calculator
- Multiplication table
- Sum of first N natural numbers
- Factorial
- Fibonacci
- Prime check
- Reverse / palindrome / sum of digits
- Nested-loop pattern
- String traversal
- Temperature conversion
- Function-based calculator and factorial

Output-based concepts to remember

- input() returns str unless explicitly converted.
- range(stop) excludes stop; range(1, 5) gives 1, 2, 3, 4.
- 10 // 3 is 3; 10 % 3 is 1; 2 ** 3 is 8.
- 'ha' * 3 produces 'hahaha'.
- A function without return gives None.
- break exits the loop; continue skips to the next iteration.
- Indentation decides which statements belong to a block.

Exam answer presentation tips

- Start with a precise definition, then explain in short bullet points.
- Write syntax before the program when asked to explain a statement.
- For a program question, write the algorithm, code and sample output.
- Keep indentation exactly as shown. Use meaningful variable names.
- For flowcharts, label Yes/No branches and use standard symbols.
- Do not write a PYQ year unless you have checked the actual paper.

Final reminder: This PDF is a comprehensive learning and practice resource based on the supplied Unit 4 topic list. Match it against the latest official ACS101 syllabus issued by your college/university, because course outlines and paper patterns can change.